

Lernskript

Shellprogrammierung, reguläre Ausdrücke, grep, sed und Textverarbeitung

Prüfungsvorbereitung mit erklärten Folien, Originalaufgaben und einem vollständigen Prüfungsbeispiel

Fachlich geprüfte und korrigierte Fassung aus den bereitgestellten Lehrveranstaltungsunterlagen

Prüfungsfokus: ein Shellskript mit ungefähr 20–30 Zeilen, Argumentprüfung, Dateioperationen und Textverarbeitung; grep und sed können als getrennte Aufgaben oder innerhalb des Skripts vorkommen.

Dieses Skript ist so aufgebaut, dass jedes Kapitel zuerst die Idee erklärt, anschließend prüfungsnahe Beispiele zeigt und danach Originalaufgaben aus den PDFs anbietet. Die eingebetteten Abbildungen sind Folien aus den Unterlagen und stehen jeweils direkt bei dem Thema, das sie veranschaulichen.

Hinweis zur Schreibweise: Die Folien verwenden teilweise ältere Formen wie Backticks (`...`) oder sed -r. Im Erklärungstext stehen die heute üblichen Formen \$(...) und sed -E. Alle ausführbaren Skriptbeispiele sind für Bash (#!/bin/bash) auf einem GNU/Linux-System formuliert; wichtige Abweichungen und GNU-spezifische Stellen werden im Text gekennzeichnet.

Inhaltsverzeichnis

- [1 Shell-Grundlagen](#)
- [2 Aufbau eines Shellskripts](#)
- [3 Tests, Bedingungen und Verzweigungen](#)
- [4 Schleifen und zeilenweises Dateilesen](#)
- [5 Rechnen in der Shell](#)
- [6 Reguläre Ausdrücke](#)
- [7 grep sicher anwenden](#)
- [8 sed zum Suchen, Ersetzen und Löschen](#)
- [9 Werkzeugkasten Textverarbeitung](#)
- [10 Prüfungsmuster und vollständiges Beispiel](#)
- [A Kompakte Befehlsübersicht](#)
- [B Quellenübersicht](#)

1 Shell-Grundlagen

Die Shell ist gleichzeitig Kommandointerpreter und kleine Programmiersprache. Sie liest eine Kommandozeile, ersetzt Variablen und Wildcards, richtet Ein- und Ausgabe ein und startet anschließend das gewünschte Programm. Für die Prüfung ist wichtig, diese Verarbeitungsschritte auseinanderzuhalten: Viele Fehler entstehen nicht im Programm `grep` oder `sed`, sondern schon vorher durch die Shell.

Was kann die Shell? (1)

- Shell ist ein Kommandointerpreter
 - `sh`, `csh`, `ksh`, `bash` oder ein selbst geschriebener Kommandozeileninterpreter
- Kommandos entgegennehmen
- Kommando suchen
- Ein- und Ausgabe des Kommandos im Shellfenster durchführen
- Ersetzen von Sonderzeichen und Befehlssubstitution
- Ablaufsteuerung (Anweisungen, Schleifen, etc. → Shellprogrammierung)

2

Abbildung 1: Aufgaben der Shell. Quelle: *Slides - Basics (WH).pdf*, Folie 2.

1.1 Kommando, Argumente und Rückgabewert

Ein Kommando besteht typischerweise aus **Programmname**, **Optionen** und **Argumenten**. Beispiel:

```
grep -E -i -n 'error|warning' server.log
```

• `grep` ist das Programm.

• `-E`, `-i` und `-n` sind Optionen.

• `error|warning` ist der Suchausdruck.

• `server.log` ist die Eingabedatei.

Jedes Programm liefert einen Statuscode. Nach Unix-Konvention bedeutet 0 Erfolg; die Bedeutung anderer Werte legt das jeweilige Programm fest. Der Status des unmittelbar zuvor ausgeführten Befehls steht in `$_`. `grep` liefert 0 bei mindestens einem Treffer, 1 bei keinem Treffer und einen Wert größer als 1 bei einem Fehler.

1.2 Standard-Eingabe, Standard-Ausgabe und Fehlerausgabe

Unix-Programme arbeiten mit drei Standardkanälen: `stdin` (0), `stdout` (1) und `stderr` (2). Durch

Umleitungen kann man diese Kanäle mit Dateien verbinden. Das ist die Grundlage fast aller Textverarbeitungsaufgaben.

Schreibweise	Bedeutung	Beispiel
< datei	Datei als Eingabe	sort < namen.txt
> datei	Ausgabe neu schreiben; vorhandener Inhalt wird überschrieben	grep ERROR log.txt > fehler.txt
>> datei	Ausgabe anhängen	echo "fertig" >> protokoll.txt
2> datei	nur Fehlermeldungen umleiten	ls /nichtda 2> fehler.log
2>&1	stderr auf dasselbe Ziel wie stdout legen	befehl > alles.log 2>&1
> /dev/null	Ausgabe verwerfen	befehl > /dev/null

Ein- / Ausgabeumleitung (1)

- Eingabeumleitung
 - Programm liest aus Datei, nicht von Tastatur
 - Umleitung erfolgt durch ,<‘
 - Numerischer Wert: 0
- Ausgabeumleitung
 - Ergebnis wird in Datei geschrieben, nicht auf Bildschirm
 - Umleitung erfolgt durch ,>‘ bzw. ,>>‘
 - Numerischer Wert: 1

4

Abbildung 2: Ein- und Ausgabeumleitung. Quelle: Slides - Basics (WH).pdf, Folie 4.

Prüfungsfalle: > leert eine vorhandene Datei vor dem Schreiben. >> hängt an. Vor Schleifen wird eine Ausgabedatei häufig einmal mit > "\$out" geleert; innerhalb der Schleife wird dann mit >> angehängt.

1.3 Pipes: kleine Programme kombinieren

Eine Pipe | verbindet stdout des linken Befehls mit stdin des rechten Befehls. Dadurch kann ein Problem in kleine Schritte zerlegt werden: zuerst filtern, dann Felder ausschneiden, anschließend sortieren und zählen.

```
grep -E '^(RJ|WB)' fahrten.txt | cut -d';' -f2 | sort | uniq -c
```

•grep wählt relevante Zeilen aus.

•cut extrahiert ein Feld.

•sort bringt gleiche Werte nebeneinander.

•uniq -c zählt aufeinanderfolgende gleiche Werte.

Pipes

- Eine Pipe verbindet Ausgabe eines Programms und Eingabe eines zweiten Programms über einen temporären Puffer
- Mehrere Kommandos können hintereinander geschaltet werden

```
osboxes@osboxes: ~/Desktop
osboxes@osboxes:~/Desktop$ dmesg | grep usb
[ 3.399409] usbcore: registered new interface driver usbfs
[ 3.399414] usbcore: registered new interface driver hub
[ 3.399419] usbcore: registered new device driver usb
[ 3.801842] usb usb1: New USB device found, idVendor=1d6b, idProduct=
[ 3.801852] usb usb1: New USB device strings: Mfr=3, Product=2, Serial=
[ 3.801857] usb usb1: Product: OHCI PCI host controller
[ 3.801862] usb usb1: Manufacturer: Linux 4.2.0-16-generic ohci_hcd
[ 3.801865] usb usb1: SerialNumber: 0000:00:06.0
[ 4.480333] usb 1-1: new full-speed USB device number 2 using ohci-pc
[ 4.736482] usb 1-1: New USB device found, idVendor=80ee, idProduct=0021
[ 4.736491] usb 1-1: New USB device strings: Mfr=1, Product=3, SerialNumber=0
[ 4.736497] usb 1-1: Product: USB Tablet
[ 4.736501] usb 1-1: Manufacturer: VirtualBox
[ 4.762817] usbcore: registered new interface driver usbhid
[ 4.762825] usbhid: USB HID core driver
[ 4.768414] input: VirtualBox USB Tablet as /devices/pci0000:00/0000:00:06.0/usb1/1-1/1-1:1.0/0003:80EE:0021.0001/input/input6
[ 4.768646] hid-generic 0003:80EE:0021.0001: input,hidraw0: USB HID v1.1 Mouse [VirtualBox USB Tablet] on usb-0000:00:06.0-1/input0
osboxes@osboxes:~/Desktop$
```

Abbildung 3: Grundidee einer Pipe. Quelle: Slides - Basics (WH).pdf, Folie 8.

Eine unnötige Verwendung von cat ist meist vermeidbar: grep Muster datei ist klarer als cat datei | grep Muster. In Prüfungsaufgaben sind beide Varianten oft akzeptiert, die direkte Dateiangabe ist aber leichter zu lesen.

1.4 Globbing ist nicht dasselbe wie eine Regular Expression

Wildcards der Shell werden vor dem Programmstart in Dateinamen umgewandelt. Das Muster *.txt bedeutet: alle Dateinamen, die auf .txt enden. Bei grep oder sed beschreibt eine Regular Expression dagegen Zeichen in Textzeilen. Dort würde ein ähnliches Muster beispielsweise .*\.txt\$ lauten.

Ziel	Shell-Globbing	Regular Expression
beliebig viele Zeichen	*	.*
genau ein Zeichen	?	.
Dateien mit Endung .txt	*.txt	\.txt\$
Zeichen aus Menge	[abc]	[abc]

Globbering - Pfadnamenerweiterung

- Um an ein Kommando eine Liste von Dateinamen zur Verarbeitung zu übergeben, können diese vollständig angegeben werden. Oder man gibt ein oder mehrere „Suchmuster“ (Wildcards) an, die von der aktuellen Shell in passende Dateinamen umgewandelt werden.
- Die Shell expandiert zuerst alle Suchmuster zu einer Liste von passenden Dateinamen ("filename globbing"), setzt diese dann anstelle der Suchmuster ein und führt schließlich das Kommando aus.
- Unabhängig vom auszuführenden Kommando wird die Dateinamenexpansion immer von der Shell durchgeführt, BEVOR ein Kommando gestartet wird.

12

Abbildung 4: Die Shell erweitert Wildcards vor dem Start des Kommandos. Quelle: Slides - Basics (WH).pdf, Folie 12.

1.5 Variablen und Quoting

Eine Zuweisung enthält keine Leerzeichen um das Gleichheitszeichen. Der Zugriff erfolgt mit \$. Variablen sollten fast immer in doppelte Anführungszeichen gesetzt werden, damit Leerzeichen und Wildcards im Wert nicht erneut von der Shell zerlegt werden.

```
datei='Mein Protokoll.txt'  
anzahl=$(wc -l < "$datei")  
echo "$datei enthält $anzahl Zeilenumbrüche"
```

Schreibweise

'Text \$HOME'
"Text \$HOME"
\$(kommando)
*

Wirkung

Alles bleibt wörtlich; keine Variablen- oder Kommandosubstitution.
Variablen und \$(...) werden ersetzt; Leerzeichen bleiben Teil eines Arguments.
Ausgabe eines Kommandos wird als Text eingesetzt.
Der Backslash nimmt einem Metazeichen seine Sonderbedeutung.

Quoting

- Verschiedene Hochkomma haben Spezialbedeutung
 - `".."` Keine Ersetzung der (`*`, `?`, `[]`) Wildcards, jedoch SHELL-Variable mit `$` werden ersetzt
 - `'...'` unterdrückt jede Substitution
 - ``...`` Kommando zwischen Back-Quotes, wird ausgeführt und Ergebnis liefert Wert der Variablen



```
osboxes@osboxes: ~/Desktop
osboxes@osboxes:~/Desktop$ VAR="Hello World!"
osboxes@osboxes:~/Desktop$ echo $VAR
Hello World!
osboxes@osboxes:~/Desktop$ echo '$VAR'
$VAR
osboxes@osboxes:~/Desktop$ VAR=`pwd`
osboxes@osboxes:~/Desktop$ echo $VAR
/home/osboxes/Desktop
osboxes@osboxes:~/Desktop$ echo ${VAR}/bin
/home/osboxes/Desktop/bin
```

20

Abbildung 5: Quoting in der Shell. Quelle: Slides - Basics (WH).pdf, Folie 20.

[Zurück zum Inhaltsverzeichnis](#)

2 Aufbau eines Shellskripts

2.1 Shebang, Dateirechte und Aufruf

Die erste Zeile legt den Interpreter fest. Dieses Lernskript verwendet Bash, weil spätere Beispiele Bash-Syntax wie `$\r` und `[[ ... ]]` benutzen. Ein Skript mit `#!/bin/sh` darf nur Syntax verwenden, die der dort gestartete sh-Interpreter unterstützt. Danach muss die Datei ausführbar sein oder explizit mit dem passenden Interpreter gestartet werden.

```
#!/bin/bash
```

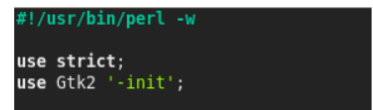
```
echo "Hallo aus dem Skript"
```

```
# Danach im Terminal ausführen (nicht in beispiel.sh):  
chmod +x beispiel.sh  
./beispiel.sh
```

Shebang Line

```
#!/bin/sh
```

```
#!/bin/bash
```



```
#!/usr/bin/perl -w  
use strict;  
use Gtk2 '-init';
```

Diese Zeile weist das Betriebssystem an, diese Datei mit dem Interpreter-Programm `/bin/sh`, in diesem Fall also der Standard-Unix-Shell, auszuführen.

6

Abbildung 6: Bedeutung der Shebang-Zeile. Quelle: Slides - Intro.pdf, Folie 6.

2.2 Stellungsparameter und spezielle Parameter

Argumente werden beim Aufruf an das Skript übergeben. `$1` ist das erste Argument, `$2` das zweite usw. Die Anzahl steht in `$#` . Besonders wichtig ist der Unterschied zwischen `"$@"` und `"$*"`: `"$@"` erhält jedes Argument als eigenes Wort und ist daher in Schleifen fast immer richtig.

Parameter	Bedeutung	Typischer Einsatz
<code>\$0</code>	Name des Skripts	Fehlermeldung: Usage: <code>\$0</code> datei
<code>\$1, \$2, ...</code>	einzelne Argumente	Eingabe- und Ausgabedatei
<code>\$#</code>	Anzahl der Argumente	<code>if ["\$#" -ne 2]</code>
<code>\$@</code>	alle Argumente, getrennt	<code>for arg in "\$@"</code>
<code>\$?</code>	Status des letzten Befehls	nach <code>grep/test</code> prüfen
<code>\$\$</code>	Prozess-ID der Shell	PID anzeigen; Tempdateien mit <code>mktemp</code>

Shell Spezielle Parameter

Den sogenannten speziellen Parametern kann kein Wert zugewiesen werden und sie werden von Der Shell speziell behandelt:

\$*

Expandiert zu den Stellungsparametern. Werden doppelte Anführungszeichen benutzt ("\$*"), so wird zu einem einzigen Wort expandiert. Die Werte der Stellungsparameter sind durch das erste Zeichen von \$IFS (normalerweise das Leerzeichen) getrennt (also "\$1 \$2 \$3 ...").

\$@

Expandiert zu den Stellungsparametern. Werden doppelte Anführungszeichen benutzt ("\$@"), so expandiert jeder Parameter zu einem eigenen Wort (also "\$1" "\$2" "\$3" ...).

\$#

Expandiert zur Anzahl der Stellungsparameter.

\$?

Expandiert zum Rückgabewert des letzten, im Vordergrund ausgeführten Befehls.

\$\$

Expandiert zur PID der Shell (auch wenn in einer Subshell verwendet).

\$0

Expandiert zum Namen der Shell oder des Shell-Skripts.

3

Abbildung 7: Spezielle Shellparameter. Quelle: Slides - Intro.pdf, Folie 3.

2.3 Interaktive Eingabe und Rückgabewerte

Mit read werden Werte von stdin gelesen. Die robuste Standardform ist read -r, weil Backslashes dann nicht als Escape-Zeichen behandelt werden. exit 0 signalisiert Erfolg; ein Wert von 1 bis 255 signalisiert einen Fehler oder einen anderen nicht erfolgreichen Zustand.

```
printf 'Dateiname: '
read -r datei

if [ ! -f "$datei" ]; then
echo "Fehler: $datei existiert nicht." >&2
exit 1
fi
```

2.4 Ein prüfungstaugliches Grundgerüst

```
#!/bin/bash

if [ "$#" -ne 2 ]; then
echo "Aufruf: $0 eingabe ausgabe" >&2
exit 1
fi

eingabe=$1
ausgabe=$2

if [ ! -f "$eingabe" ]; then
echo "Fehler: Datei $eingabe nicht gefunden." >&2
exit 1
fi

if [ "$eingabe" = "$ausgabe" ]; then
```

```
printf 'Fehler: Ein- und Ausgabepfad sind gleich.\n' >&2
exit 1
fi
if ! : > "$ausgabe"; then
printf 'Fehler: Ausgabedatei nicht beschreibbar.\n' >&2
exit 1
fi
```

Ab hier folgt die eigentliche Verarbeitung.

Dieses Muster deckt typische Prüfungsanforderungen ab: Argumentzahl, Variablenzuweisung, Existenztest, getrennte Ein- und Ausgabepfade, Fehlermeldung auf stderr und eine geprüfte Initialisierung der Ausgabedatei. Der Befehl `:` tut nichts; seine Umleitung prüft hier, ob die Datei geöffnet werden kann.

Originalaufgabe 2.1: mmv - mehrere Dateieindungen ändern

Quelle: Shellprogrammierbeispiele.pdf, Seite 1. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Erstellen Sie ein Shellsript `mmv` (= multiple move), das die Extensions von Filenamen, die sich im aktuellen Arbeitsverzeichnis befinden, ändert. Überprüfen Sie auch die Anzahl der Argumente.

Beispiel: `mmv htm html` - Umbenennen aller Files mit der Extension `.htm` in `.html`.

Anmerkung: Unterverzeichnisse sind nicht zu ändern.

[Zurück zum Inhaltsverzeichnis](#)

3 Tests, Bedingungen und Verzweigungen

3.1 test und [...]

test Bedingung und [Bedingung] sind zwei Schreibweisen für dasselbe Prinzip. Sie erzeugen normalerweise keine Ausgabe: Status 0 bedeutet wahr, Status 1 falsch; ein Wert größer als 1 weist auf einen Aufruf- oder Syntaxfehler hin. Bei der Klammerform sind die Leerzeichen nach [und vor] zwingend.

```
if [ "$#" -eq 0 ]; then
echo "Keine Argumente übergeben."
else
echo "Anzahl: $#"
```

If Anweisung

- test Bedingung oder [Bedingung]
 - Prüft Bedingung, liefert 0 falls true, 1 falls false
- if – then – else

```
– if bedingung
  then
    kommandos
  fi

– if bedingung
  then
    kommandos
  else
    kommandos
  fi

– if bedingung1
  then
    kommandos
  elif bedingung2
  then
    kommandos
  elif .....
  fi
```

8

Abbildung 8: Aufbau einer if-Anweisung. Quelle: Slides - Intro.pdf, Folie 8.

3.2 Zahlen, Zeichenketten und Dateien prüfen

Bereich	Operator	Bedeutung
Zahlen	-eq -ne	gleich / ungleich
Zahlen	-lt -le	kleiner / kleiner oder gleich
Zahlen	-gt -ge	größer / größer oder gleich
Strings	= !=	gleich / ungleich
Strings	-z / -n	leer / nicht leer
Dateien	-e	existiert
Dateien	-f / -d	reguläre Datei / Verzeichnis
Dateien	-r -w -x	lesbar / schreibbar / ausführbar
Dateien	-s	existiert und ist nicht leer

Testoptionen auf Dateien



- Nur für Dateien/Verzeichnisse verwendbar
- Existiert Datei/VZ nicht, wird automatisch false zurückgeliefert.

Syntax: „test option dateiname“

-d	Prüft, ob es sich bei der Datei um ein Verzeichnis handelt
-e	Prüft, ob die angegebene Datei im Dateisystem existiert
-f	Prüft, ob die angegebene Datei eine reguläre (normale) Datei ist
-r	Prüft, ob der aktuelle Benutzer Leserechte auf der Datei besitzt
-w	Prüft, ob der aktuelle Benutzer Schreibrechte auf der Datei besitzt
-x	Prüft, ob der aktuelle Benutzer Exekuterechte auf der Datei besitzt
-s	Prüft, ob die Datei nicht leer ist
	u.v.a.m

12

Abbildung 9: Wichtige Dateitests. Quelle: Slides - Intro.pdf, Folie 12.

Immer quotes: Schreibe `[ -n "$name" ]` statt `[ -n $name ]`. Ohne Anführungszeichen kann ein leerer oder mehrteiliger Wert die Anzahl der Argumente von test verändern.

3.3 Mehrere Fälle mit if, elif und else

```
if [ ! -e "$pfad" ]; then
echo "Pfad existiert nicht."
elif [ -d "$pfad" ]; then
echo "Pfad ist ein Verzeichnis."
elif [ -f "$pfad" ]; then
echo "Pfad ist eine reguläre Datei."
else
echo "Anderer Dateityp."
fi
```

3.4 Kurzformen mit && und ||

Bei `A && B` wird B nur ausgeführt, wenn A den Status 0 liefert. Bei `A || B` wird B bei jedem Status ungleich 0 ausgeführt. Das kann sowohl eine nicht erfüllte Bedingung als auch ein technischer Fehler sein. Für kurze, eindeutig interpretierbare Prüfungen sind diese Formen praktisch; sonst ist ein if- oder case-Block sicherer.

```
[ -f "$datei" ] && echo "Datei vorhanden"
grep -qE '^ERROR' "$datei" && echo "ERROR-Zeile gefunden"
```

Ergebnisabhängige Befehlsausführung

- Wenn ein Befehl ausgeführt wurde gibt er standardmäßig einen Fehlercode zurück
 - Bei erfolgreicher Ausführung ist dies der Wert 0
 - Bei einem Fehler der Wert > 0
 - Abfrage des Fehlercodes mit **echo \$?** möglich
- Fehlercode wird verwendet um Befehle abhängig von dessen Ergebnis miteinander zu verknüpfen
 - `command1 && command2`
 - Bei dieser Verknüpfung wird `command2` nur ausgeführt wenn der `command1` erfolgreich ausgeführt wurde, also Fehlercode 0 geliefert hat. (Befehle sind mit einem logischen UND (AND) verknüpft.)
 - `command1 || command2`
 - Bei dieser Verknüpfung wird der `command2` nur ausgeführt wenn der `command1` einen Fehler verursacht hat, also den Fehlercode >0 geliefert hat. (Befehle mit einem logischen ODER (OR) verknüpft.)

20

Abbildung 10: Ergebnisabhängige Befehlsausführung. Quelle: Slides - Intro.pdf, Folie 20.

Originalaufgabe 3.1: Prüfungen am Skriptanfang

Quelle: Alter Kurztest(1).pdf, Teilaufgaben a-c. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Dem Shellsript soll die Datei `essensplan.txt` als Argument übergeben werden. Aufruf: `./tagesbedarf.sh
essensplan.txt gesamtTagesbedarf.txt nmittel.txt`

Es ist zu überprüfen, ob die Anzahl der übergebenen Argumente stimmt und ob die Datei `essensplan.txt` überhaupt existiert. Falls die Datei nicht existiert, soll eine Fehlermeldung ausgegeben werden.

Ebenso ist zu überprüfen, ob die Textdatei unter Windows oder Linux erstellt worden ist. Wenn sich CRLFs in der Textdatei befinden, soll die Textdatei mit dem Programm `dos2unix` entsprechend überschrieben werden.

[Zurück zum Inhaltsverzeichnis](#)

4 Schleifen und zeilenweises Dateilesen

4.1 for-Schleifen

Eine for-Schleife ist geeignet, wenn eine bekannte Liste abgearbeitet wird: Argumente, Dateinamen oder durch IFS getrennte Werte. Bei Dateinamen sollte man direkt über ein Globbing-Muster iterieren und nicht die Ausgabe von ls parsen.

```
for datei in *.txt; do
[ -f "$datei" ] || continue
echo "Bearbeite: $datei"
done
```

For-Anweisung

- Mit Hilfe der for-Schleife kann man eine Anweisungsfolge in der SHELL für unterschiedliche Belegungen einer Variablen ausführen.

```
– for selector in liste
do
    kommandofolge
done
– for selector
do
    kommandofolge
done
```

15

Abbildung 11: Syntax der for-Schleife. Quelle: Slides - Intro.pdf, Folie 15.

4.2 Dateien zeilenweise lesen

Die Form `while IFS= read -r zeile` erhält Leerzeichen am Zeilenanfang und Backslashes. Der Zusatz `|| [ -n "$zeile" ]` verarbeitet auch eine letzte Zeile ohne abschließenden Zeilenumbruch. Die Konstruktion `for word in $(cat datei)` zerlegt an Leerzeichen und eignet sich nicht für ganze Zeilen.

```
while IFS= read -r zeile || [ -n "$zeile" ]; do
printf 'Zeile: %s\n' "$zeile"
done < "$datei"
```

Zeilenweises Lesen einer Datei

```
#!/bin/sh
for line in `cat file.txt`
do
    echo $line
done
```

Ausgabe:

```
Das
ist
Zeile
1
Das
ist
Zeile
2
```

```
#!/bin/sh
while read line
do
    echo $line
done < file.txt
```

Ausgabe:

```
Das ist Zeile 1
Das ist Zeile 2
```

17

Abbildung 12: Vergleich von for und while beim Dateilesen. Quelle: Slides - Intro.pdf, Folie 17.

4.3 Mehrere Felder mit IFS einlesen

Für strukturierte Daten wird zuerst die vollständige Zeile gelesen. Dadurch geht auch eine letzte Zeile ohne LF nicht verloren, selbst wenn einzelne Felder leer sind. Ein zweiter read-Aufruf zerlegt die Zeile mit IFS; <<< ist ein Bash-Here-String.

```
while IFS= read -r zeile || [ -n "$zeile" ]; do
IFS=';' read -r name wert status <<< "$zeile"
[ -n "$wert" ] || wert=0
printf '%s -> %s (%s)\n' "$name" "$wert" "$status"
done < daten.txt
```

IFS sollte möglichst nur für einen einzelnen read-Aufruf oder einen klar begrenzten Block gesetzt werden. Eine globale Änderung kann später überraschende Zerlegungen verursachen.

4.4 Kommandosubstitution und sichere Dateinamen

Mit \$(...) wird die Standardausgabe eines Kommandos als einzelner Variablenwert übernommen; abschließende Zeilenumbrüche werden entfernt. Für einen einzelnen Wert wie das Ergebnis von basename ist das passend. Dateilisten sollten dagegen mit Globbing, Arrays oder while read verarbeitet werden, nicht durch Wortzerlegung einer Kommandosubstitution.

```
basis=$(basename -- "$datei" .txt)
neuer_name="${basis}.dat"
mv -- "$datei" "$neuer_name"
```

Originalaufgabe 4.1: sizes

Quelle: Shell_Programmierung_Verwendung_Schleifen.pdf, Seite 1. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Erstellen Sie ein Shellsript `sizes`, das für ein gegebenes Verzeichnis die Größen aller darin enthaltenen Files und Unterverzeichnisse berechnet. Ein Abstieg in diese Unterverzeichnisse muss nicht erfolgen.

a) Die richtige Anzahl der übergebenen Argumente ist zu prüfen.

b) Pfad und Name des zu analysierenden Verzeichnisses werden als Argument übergeben. Es ist zu prüfen, ob dieses Verzeichnis existiert.

c) Die Analyse wird erleichtert, wenn Sie direkt in das gegebene Verzeichnis wechseln (`cd`).

d) Die Größe eines Files bzw. Verzeichnisses kann mit einer `for`-Schleife und Kommandosubstitution ermittelt werden. In der Vorlage wird dazu `ls -ld ... | cut ...` verwendet.

e) Ausgabe: Size of Directories in VZ: ... is ... Bytes; Size of Files in VZ: ... is ... Bytes.

Originalaufgabe 4.2: `bplist` und verschachtelte Trennzeichen

Quelle: Angabe `bplist` - Beispiel 3(1).pdf, Seiten 4-5. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Der File `bplist` enthält pro Satz als erstes Feld eine Institution und als zweites Feld die Namen von einem oder mehreren Studierenden. Institution und Namen sind durch Strichpunkt getrennt; mehrere Namen sind durch Beistriche getrennt.

Erstellen Sie ein Shellsript `bp`, das einen File `leute` erzeugt, der eine Liste aller Studierenden in der Form „Nachname Vorname“ enthält.

Empfohlene Vorgangsweise: IFS auf Strichpunkt setzen; `bplist` satzweise mit `institution` und `personen` einlesen; innerhalb der `while`-Schleife IFS auf Beistrich setzen; mit einer `for`-Schleife die einzelnen Namen verarbeiten; Vorname und Nachname mit `cut`/Kommandosubstitution extrahieren; danach IFS wieder auf Strichpunkt setzen.

```
click for knowledge GmbH;Monika Freudenberger  
emergentec;Georg Summer,Andreas Heinzel,Eva-Maria Forstner
```

```
TU Graz;Maria Fischer,Stephan Pabinger,Tanja Grill
```

[Zurück zum Inhaltsverzeichnis](#)

5 Rechnen in der Shell

5.1 Integerarithmetik mit `$((...))`

Für ganze Zahlen verwendet die Shell arithmetische Expansion. Das Ergebnis kann direkt ausgegeben oder einer Variablen zugewiesen werden. Variablennamen dürfen innerhalb von `$((...))` ohne `$` geschrieben werden.

```
x=$((9 / 3 * 10))
summe=$((summe + wert))
rest=$((anzahl % 2))
echo "$x"
```

Division mit ganzen Zahlen schneidet Nachkommastellen ab: `7 / 2` ergibt 3. Eingabewerte müssen vor der Rechnung als Integer validiert werden. Die Schreibweise `10#$wert` erzwingt bei Bash eine dezimale Interpretation, sodass zum Beispiel 08 nicht als ungültige Oktalzahl behandelt wird.

5.2 Dezimalzahlen mit `bc`

Für Dezimalrechnungen kann `bc` verwendet werden. Der Ausdruck wird über `stdin` an `bc` übergeben. `scale` legt bei Divisionen die Zahl der Nachkommastellen des Ergebnisses fest; es ist keine allgemeine Garantie, dass jede andere Operation exakt so viele Stellen ausgibt.

```
anz=31.2
ergebnis=$(printf 'scale=2; %s / 3\n' "$anz" | bc)
echo "$ergebnis"
```

Quelle dieser Rechenformen: Rechnen in der Shell.pdf.

5.3 Typisches Muster: Summe und Anzahl

```
summe=0; anzahl=0

while IFS= read -r zeile || [ -n "$zeile" ]; do
case $zeile in
*.*.*|:.*|:*) printf 'Ungültige Datenzeile: %s\n' "$zeile" >&2; exit 1 ;;
esac
name=${zeile%.*}; wert=${zeile#*.}
case $wert in
*[^0-9]*) printf 'Ungültige Zahl: %s\n' "$wert" >&2; exit 1 ;;
esac
summe=$((summe + 10#$wert)); anzahl=$((anzahl + 1))
done < "$datei"

printf 'Summe: %d\nAnzahl: %d\n' "$summe" "$anzahl"
```

Originalaufgabe 5.1: curriculum

Quelle: Shell_Programmierung_Verwendung_Schleifen.pdf, Seiten 2-3. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Der File `curr` zeigt ein Curriculum. Bei jedem Fach steht die Anzahl der Wochenstunden, durch Strichpunkt getrennt, in jedem der sechs Semester dabei. Beispiel: Computer Science Introduction;4;0;0;0;0.

Erstellen Sie ein Shellskript `curriculum`, das die Anzahl der Wochenstunden pro Semester berechnet und an den File `curr` anhängt: 1.Semester: XX Wochenstunden ... bis 6.Semester.

Überprüfen Sie, ob der File `curr` existiert. Er wird nicht als Argument an das Skript übergeben.

Achtung: Leere Felder sind keine gültigen Integerwerte.

[Zurück zum Inhaltsverzeichnis](#)

6 Reguläre Ausdrücke

Reguläre Ausdrücke beschreiben Mengen von Zeichenketten. Sie beantworten nicht nur „kommt dieses Wort vor?“, sondern auch „beginnt die Zeile mit zwei Großbuchstaben?“, „steht am Ende eine Zahl?“ oder „passt die gesamte Zeile zu einem vorgegebenen Format?“. `grep` sucht damit, `sed` verwendet sie zum gezielten Bearbeiten. Die folgenden Syntaxbeispiele verwenden Extended Regular Expressions (ERE).

Regular Expressions

- Regular expressions (regex) (dt. reguläre Ausdrücke), sind eine leistungsstarke, flexible und effiziente Methode zum Definieren von Suchmustern zum Stringvergleich in Texten
- Stellen umfangreiche Notation für den Mustervergleich zur Verfügung:
 - Suche nach bestimmten Zeichenmustern in Texten
 - Validieren von Text, um sicherzustellen, dass er einem vordefinierten Muster entspricht (E-Mail-Adresse, IP-Adressen);
 - Darauf aufsetzend: Extrahieren, Bearbeiten, Ersetzen oder Löschen von Textzeichenfolgen in Editoren bzw. in Programmiersprachen
- Für viele Anwendungen, die mit Zeichenfolgen arbeiten oder große Textblöcke analysieren, sind reguläre Ausdrücke unverzichtbar.
- Einfacher Anwendungsfall von regulären Ausdrücken sind die Wildcards (=Metacharakter) der Shell.

2

Abbildung 13: Einsatzgebiete regulärer Ausdrücke. Quelle: Reguläre Ausdrücke.pdf, Folie 2.

6.1 Literale und Metazeichen

Normale Zeichen stehen für sich selbst. Metazeichen steuern das Muster. Soll ein Metazeichen wörtlich gesucht werden, wird es meist mit einem Backslash geschützt: `\.` findet einen echten Punkt.

Regular Expressions – Syntax

Metazeichen

Metazeichen sind Zeichen, die nicht für sich selbst stehen, sondern eine besondere Bedeutung haben.

Metazeichen	Bedeutung
^	Anker für Anfang einer Zeile
\$	Anker für Ende einer Zeile
? + * { }	Quantifikatoren oder Wiederholungsfaktoren
\	Metazeichen verliert seine spezielle Bedeutung durch Voranstellen des Backslashes \
. []	Zeichenklassen
&	Backreference (hier nicht behandelt)
()	Gruppierung Unterausdrücke
 	Alternative

6

Abbildung 14: Übersicht wichtiger Metazeichen. Quelle: Reguläre Ausdrücke.pdf, Folie 6.

Element	Bedeutung	Beispiel
.	ein beliebiges Zeichen innerhalb der aktuellen Zeile	a.c findet abc und a-c
[...]	ein Zeichen aus einer Menge	[ABC]
[^...]	ein Zeichen nicht aus der Menge	[^0-9]
^ / \$	Zeilenanfang / Zeilenende	^ERROR\$
* + ?	0..n / 1..n / 0..1 Wiederholungen	[0-9]+
{n,m}	Wiederholungsbereich	[A-Z]{2,4}
(...)	gruppiert einen Unterausdruck	(ab)+
	Alternative	rot blau

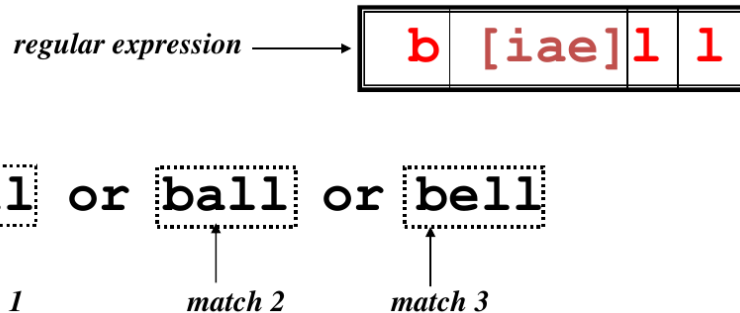
6.2 Punkt und Zeichenklassen

Der Punkt steht für genau ein Zeichen innerhalb der aktuellen Textzeile. Eine Zeichenklasse steht ebenfalls für genau ein Zeichen, allerdings nur aus der angegebenen Menge. Bereiche wie [a-z] können vom Gebietsschema abhängen; POSIX-Klassen wie [:alpha:] sind für Kategorien meist verlässlicher.

Regular Expressions – Syntax

Zeichenklasse

- Zeichenklassen können mit eckigen Klammern `[]` spezifiziert werden.
- Ein einziges Zeichen aus dem Klammerausdruck wird auf Übereinstimmung herangezogen.



8

Abbildung 15: Zeichenklassen. Quelle: Reguläre Ausdrücke.pdf, Folie 8.

POSIX-Zeichenklassen sind in grep/sed besonders portabel: `[:digit:]` für Ziffern, `[:alpha:]` für Buchstaben, `[:space:]` für Whitespace, `[:alnum:]` für Buchstaben oder Ziffern. Die äußeren eckigen Klammern gehören zur Syntax.

6.3 Anker: Anfang und Ende festlegen

Ohne Anker genügt ein Treffer irgendwo in der Zeile. Mit `^` und `$` kann die Position festgelegt werden. Das Muster `^[0-9]+$` akzeptiert nur Zeilen, die vollständig aus Ziffern bestehen.

Regular expressions: Anker

Beispiele:

<code>^D</code>	Zeile, die mit D beginnt
<code>D\$</code>	Zeile, die mit D endet
<code>^D\$</code>	Zeile, die nur D enthält
<code>^.\$</code>	Zeile, die nur einen Buchstaben enthält
<code>^...\$</code>	Zeile, die nur drei Buchstaben enthält
<code>^e..</code>	Zeile, die mit e beginnt und mindestens noch zwei Zeichen enthält
<code>^\.</code>	Zeile mit echtem Punkt am Anfang
<code>^\$</code>	leere Zeile

14

Abbildung 16: Beispiele für Anker. Quelle: Reguläre Ausdrücke.pdf, Folie 14.

`^D` # Zeile beginnt mit D
`D$` # Zeile endet mit D
`^$` # leere Zeile
`^[0-9]+$` # nur Ziffern; 0 und führende Nullen sind erlaubt

6.4 Quantifikatoren

Ein Quantifikator bezieht sich immer auf den unmittelbar davorstehenden Ausdruck. Bei `ca+` wird nur das `a` wiederholt; bei `(ca)+` wird die gesamte Gruppe wiederholt.

Regular Expressions – Syntax Quantifikatoren

Quantifikator	Beschreibung
?	Der vorangegangene Ausdruck ist optional und wird maximal einmal getroffen
*	Der vorangegangene Ausdruck wird beliebig oft (aber auch keinmal vorgefunden)
+	Der vorangegangene Ausdruck wird mindestens einmal gefunden
{n}	Der vorangegangene Ausdruck wird genau n-mal gefunden
{n,}	Der vorangegangene Ausdruck wird mindestens n-mal oder öfter angetroffen
{n,m}	Der vorangegangene Ausdruck wird mindestens n-mal und maximal m-mal angetroffen
{0,m}	Der vorangegangene Ausdruck wird maximal m-mal angetroffen.

15

Abbildung 17: Quantifikatoren. Quelle: Reguläre Ausdrücke.pdf, Folie 15.

```
ca+r # car, caar, caaar, ...
ca?r # cr oder car
ca{3}r # caaar
ca{3,5}r # drei bis fünf a
[0-9]{1,2} # eine oder zwei Ziffern
```

6.5 Gruppierung und Alternativen

Runde Klammern machen mehrere Zeichen zu einer Einheit; | trennt Alternativen. Mit grep -E und sed -E wird ERE-Syntax verwendet. Bei GNU grep in BRE-Schreibweise benötigen mehrere dieser Operatoren einen Backslash; für portable und gut lesbare Lösungen ist ERE hier die sichere Wahl.

Regular Expression - Syntax Alternativen

Mehrere Alternativen können durch die Verwendung des **|** Operators unter Verwendung von Unterausdrücken **()** angegeben werden.

Beispiel:

```
I like (dogs|penguins), but not (lions|tigers).
```

findet

```
I like dogs, but not lions.  
I like dogs, but not tigers.  
I like penguins, but not lions.  
I like penguins, but not tigers.
```

19

Abbildung 18: Alternativen mit Gruppen. Quelle: Reguläre Ausdrücke.pdf, Folie 19.

Variante	Aufruf	Besonderheit
BRE	grep Muster datei	GNU grep: mehrere Operatoren mit Backslash; portable Unterschiede beachten
ERE	grep -E Muster datei	erweiterte Syntax direkt verwendbar
Fixed String	grep -F Text datei	keine Regex; alle Zeichen wörtlich

Originalaufgabe 6.1: Regexp-Übungen

Quelle: T2_RegEx_Übung.pdf, Seiten 1-2. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Verwenden Sie den folgenden Text als Datenbasis und formulieren Sie passende reguläre Ausdrücke.

- 1) Finden Sie alle Textstellen, die das Wort Engineering beinhalten.
- 2) Finden Sie alle Textstellen, die Software Engineering oder Web Engineering beinhalten.
- 3) Finden Sie alle kleingeschriebenen deutschen Umlaute ä, ü, ö.
- 4) Finden Sie alle Großbuchstaben und ÖÜÄ im Text.
- 5) Finden Sie alle Worte, die großgeschrieben sind. Das ganze Wort soll ausgegeben werden.
- 7) Finden Sie alle Worte, die mit einem Großbuchstaben beginnen und mit einem Großbuchstaben enden.
- 8) Finden Sie alle Worte in Klammern.
- 9) Finden Sie alle Worte, die genau aus zwei Großbuchstaben bestehen.
- 10) Finden Sie alle gültigen INTEGER-Zahlen.
- 11) Finden Sie alle Telefon- bzw. Faxnummern.
- 12) Finden Sie Textstellen, deren Worte durch / getrennt sind.
- 13) Finden Sie Textstellen, deren Worte durch - getrennt sind.
- 14) Finden Sie alle gültigen Zeitangaben.

Der Studiengang Software Engineering feierte 2013 sein 20-jähriges Bestehen!

Die Jubiläumsfeier fand am Samstag, 14. September 2013 um 16:00 Uhr in Hagenberg statt.

Im Vollzeit-Studium Software Engineering entscheiden sich die Studierenden zwischen Business Software und Web Engineering.

Die berufsbegleitenden Lehrveranstaltungen finden an Freitagen von 15.30 bis 19.40 Uhr

bzw. 20.30 Uhr statt.
FH OÖ Studienbetriebs GmbH
Softwarepark 11, 4232 Hagenberg/Austria
Tel.: +43 (0)50804-22001
E-Mail: se@fh-hagenberg.at
Web: www.fh-ooe.at/se

Originalaufgabe 6.2: Österreichische KFZ-Kennzeichen

Quelle: Regexp_grep_sed_Übung.pdf, Seiten 3-4. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Im File kfz befinden sich Kennzeichen; das Landeswappen wurde durch ein Blank ersetzt. Links steht die Zulassungsbehörde aus ein oder zwei Großbuchstaben. Rechts folgt ein Ziffernblock und danach ein Großbuchstabenblock.

Bei einer einstelligen Behörde folgen rechts minimal 3 und maximal 6 Zeichen; bei einer zweistelligen Behörde minimal 3 und maximal 5 Zeichen.

a) Formulieren Sie eine grep-Anweisung, die Formatierung vor dem Wappen und Gesamtlänge prüft und Treffer in ./tmp speichert.

b) Prüfen Sie anschließend, ob rechts die Reihenfolge Ziffernblock gefolgt von Buchstabenblock korrekt ist; beide Blöcke müssen vorkommen.

c) Kombinieren Sie beide grep-Anweisungen mit einer Pipe.

PE1246A

L 527DK

LL 1BC

W 12345A

I 1WE3RT

FR 1234ACS

L 3A

PE 32X

L 32UM

[Zurück zum Inhaltsverzeichnis](#)

7 grep sicher anwenden

7.1 Aufgabe und Varianten

grep gibt standardmäßig jede Zeile aus, in der das Muster mindestens einmal vorkommt. Für komplexe Prüfungsregexps ist grep -E meist die klarste Variante. grep -F ist ideal, wenn ein Suchtext Sonderzeichen enthält, aber ausdrücklich keine Regex sein soll.

Varianten von grep

Drei grep Varianten vorhanden:

- **grep**

Standardvariante mit **B**asic **R**egular **E**xpressions (BRE), lässt aber bestimmte regexs NICHT zu.

Beispiele:

Die Metazeichen `? + {} | ( )` können nicht direkt verwendet werden, sondern nur in Kombination mit einem Backslash: `\? \+ \{ \} \| \(\ \)`

- **grep -E (extended grep)**

lässt **E**xtending **R**egular **E**xpressions (ERE) zu, Metazeichen werden ohne `\` akzeptiert

- **grep -F (fast grep)**

Stark vereinfachte Version, lässt nur Strings zu, keine regulären Ausdrücke

3

Abbildung 19: grep, grep -E und grep -F. Quelle: Kommandos grep sed(1).pdf, Folie 3.

7.2 Wichtige Optionen

Option	Wirkung	Prüfungsbeispiel
-c	Anzahl passender Zeilen	grep -c ERROR log
-i	Groß-/Kleinschreibung ignorieren	grep -i linux text
-n	Zeilennummern anzeigen	grep -nE Muster datei
-o	jede nichtleere Fundstelle getrennt ausgeben	grep -oE "[0-9]+" datei
-v	nicht passende Zeilen	grep -vE "^#" config
-l	nur Dateinamen mit Treffern	grep -l ERROR *.log
-q	keine Ausgabe; nur Statuscode	grep -qE Muster datei
-E	Extended Regular Expressions	grep -E "A B"
-F	feste Zeichenkette	grep -F "a.b"

grep - Syntax

- Syntax:
`grep [-Optionen] regex [dateien]`
- Optionen:
 - c (count) gibt nur die Anzahl passender Zeilen aus
 - h (hide) unterdrückt die Ausgabe der Dateinamen
 - i (ignore) keine Unterscheidung zwischen Groß - und Kleinschreibung
 - l (list) gibt nur die Namen der Dateien aus, die passende Zeilen enthalten
 - n (number) stellt jeder Ausgabezeile die Zeilennummer aus der entsprechenden Eingabedatei voraus.
 - o (only) gibt jeden Match in einer eigenen Zeile aus
 - s (supress) unterdrückt Fehlermeldungen über nicht existierende Dateien
 - v (vice versa) gibt alle Zeilen aus, die nicht zu regex passen
 - E extended regular expressions
 - F fast grep
- Weitere Details: **man grep**

4

Abbildung 20: Syntax und Optionen von grep. Quelle: Kommandos grep sed(1).pdf, Folie 4.

7.3 Muster quoten

Regexps sollten meist in einfachen Anführungszeichen stehen. Dadurch interpretiert die Shell Zeichen wie \$, *, [oder | nicht selbst. Soll eine Variable Teil der Regex sein, sind doppelte Anführungszeichen nötig; ihr Inhalt bleibt dann Regex-Syntax. Für einen variablen, wörtlichen Suchtext ist grep -F -- "\$suchtext" die sichere Form.

```
grep -E '^[[:space:]]*ERROR:' logfile
grep -F 'a.b[c]' datei
grep -E "$ereignis[[:space:]]" wetter
```

7.4 grep in if-Anweisungen

```
grep -qE '^Hagel[[:space:]]' "$HOME/wetter"
case $? in
0) echo "Hagel gefunden" ;;
1) echo "Kein Hagel gefunden" ;;
*) echo "Fehler beim Lesen der Wetterdatei" >&2 ;;
esac
```

Für die hier verwendete einzelne Eingabedatei gilt: grep liefert 0 bei einem Treffer, 1 ohne Treffer und einen Wert größer als 1 bei einem Lesefehler. Der case-Block unterscheidet diese Fälle unmittelbar; vor \$? darf deshalb kein weiterer Befehl stehen. Bei grep -q mit mehreren Dateien kann ein früher Treffer einen späteren Dateifehler verdecken.

Originalaufgabe 7.1: Zugverkehr mit grep

Quelle: Regexp_grep_sed_Übung.pdf, Seiten 1-2. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Der File zuege enthält Station, Datum, ab/an, Uhrzeit, Dauer und Zugnummer. Trennzeichen zwischen den

Feldern sind Tabs; innerhalb der Felder stehen Leerzeichen.

- a) Wie viele Züge kommen am Wiener Westbahnhof an?
- b) Erstellen Sie eine Liste abfahrt mit allen Zügen, die am 12.05. oder 13.05. von Linz abfahren.
- c) Wie viele Züge fahren unter 1:25 nach Wien?
- d) Erstellen Sie eine Datei abfahrt_vormittag mit allen Zügen, die vor 12:00 von Linz abfahren.
- e) Erstellen Sie eine numerisch sortierte Liste verkehrsmittel, die nur RJ- und WB-Züge enthält. Gewünschte Form: RJ 765, RJ 861, WB 911, WB 913.

```
Station Datum ab/an Zeit Dauer Zugnummer
Linz/Donau Hbf 11.05.2015 ab 09:30 01:34 IC 547
Wien Hbf an 11:04
Linz/Donau Hbf 11.05.2015 ab 09:45 01:27 ICE 323
Wien Hbf an 11:12
Linz/Donau Hbf 11.05.2015 ab 10:02 01:20 WB 911
Wien Westbahnhof an 11:22
Linz/Donau Hbf 12.05.2015 ab 10:15 01:15 RJ 861
Wien Westbahnhof an 11:30
Linz/Donau Hbf 12.05.2015 ab 10:30 01:34 IC 549
Wien Westbahnhof an 12:04
Linz/Donau Hbf 12.05.2015 ab 11:02 01:20 WB 913
Wien Westbahnhof an 12:22
Linz/Donau Hbf 13.05.2015 ab 11:15 01:15 RJ 765
Wien Westbahnhof an 12:30
Linz/Donau Hbf 13.05.2015 ab 11:30 01:34 IC 693
Wien Westbahnhof an 13:04
Linz/Donau Hbf 13.05.2015 ab 11:45 01:27 ICE 21
Wien Hbf an 13:12
```

[Zurück zum Inhaltsverzeichnis](#)

8 sed zum Suchen, Ersetzen und Löschen

8.1 Das Grundmodell

sed ist ein nichtinteraktiver Stream-Editor. Es liest Zeilen, führt Befehle auf ihnen aus und schreibt das Ergebnis nach stdout. Ohne -i wird die Eingabedatei nicht verändert. Dieses Lernskript verwendet sed -E für Extended Regular Expressions; -E ist bei den üblichen GNU-/BSD-Implementierungen verfügbar, aber nicht Bestandteil jeder historischen sed-Variante.

Suchen und Ersetzen von Text mit sed

- sed verwendet regular expressions zum Suchen von Text in Files

Syntax:

```
sed 'ADDRESSs/REGEXP/REPLACEMENT/FLAGS' filename  
sed 'PATTERNs/REGEXP/REPLACEMENT/FLAGS' filename
```

- ADDRESS und PATTERNs siehe nächste Folie
- s ist die Abkürzung für substitute (s)
- / ist Trennzeichen, man kann aber auch bei Bedarf ein anderes verwenden: z.B. # oder ? oder ;
- REGEX gibt das zu suchende Muster im Text vor
- REPLACEMENT ist die Zeichenkette, mit der die gefundene Zeichenkette ersetzt wird

6

Abbildung 21: Grundsyntax der sed-Substitution. Quelle: Kommandos grep sed(1).pdf, Folie 6.

```
sed -E 's/REGEX/ERSATZ/FLAGS' eingabe > ausgabe
```

8.2 Substitution und Flags

Form

```
s/alt/neu/  
s/alt/neu/g  
s/alt/neu/2  
s/alt/neu/p  
s/alt/neu/i
```

Bedeutung

```
erstes Vorkommen pro Zeile ersetzen  
alle Vorkommen pro Zeile ersetzen  
zweites Vorkommen pro Zeile ersetzen  
geänderte Zeile ausgeben; meist mit -n  
ohne Beachtung der Groß-/Kleinschreibung (GNU sed)
```

Suchen und Ersetzen von Text mit sed

- FLAGS können wie folgt angegeben werden:
 - g** Ersetze alle gefunden Textpassagen, die zu REGEX passen, mit REPLACEMENT (ohne g wird immer nur die erste in einer Zeile ersetzt)
 - n** Kann beliebige Zahl sein, ersetzt n-tes Vorkommen von REGEX mit REPLACEMENT.
 - p** Falls eine Ersetzung durchgeführt wurde, wird die geänderte Zeile ausgegeben
 - i** verwende REGEX ohne Rücksicht auf Groß- und Kleinschreibung

8

Abbildung 22: Wichtige sed-Flags. Quelle: Kommandos grep sed(1).pdf, Folie 8.

```
sed -E 's/Linux/Linux-Unix/' mydata
sed -E 's/Linux/Linux-Unix/g' mydata
sed -E 's/Linux/Linux-Unix/2' mydata
```

8.3 Adressen: nur bestimmte Zeilen bearbeiten

Vor dem sed-Befehl kann eine Adresse stehen. Das kann eine Zeilennummer, ein Zeilenbereich oder ein Regex-Muster sein. Der folgende s-Befehl wird dann nur auf den ausgewählten Zeilen ausgeführt.

```
sed -E '1,4s/Linux/LINUX/g' mydata
sed -E '/^ics2f/s/06/16/g' besucher
sed -E '/-/s/-.*//g' mydata
```

sed - Suchen und Ersetzen: Beispiele

Beispiel 4: Suche alle Zeilen, die einen Bindestrich enthalten und entferne alle Zeichen nach dem Bindestrich bis zum Zeilenende (PATTERN)

```
sed -r '/-/s/-.*//g' mydata
```

Pattern

```
sed -r '/ich/s/€1500/€5000/g' Gehalt.dat
```

Pattern

s - Command

Instruction Guides

1. Linux Sysadmin, Linux Scripting etc.

2. Databases

3. Security (Firewall, Network, Online Security etc)

4. Storage in Linux

5. Productivity (Too many technologies to explore, not much time available)

Additional FAQs

6. Windows

Beispiel 5: Suche im Zeilenbereich 1-4 nach Linux und ersetze Linux durch LINUX

```
sed -r '1,4 s/Linux/LINUX/g' mydata
```

Addresses

s - Command

Instruction Guides

1. **LINUX** Sysadmin, **LINUX** Scripting etc.

2. Databases - Oracle, MySQL etc.

3. Security (Firewall, Network, Online Security etc)

4. Storage in Linux

5. Productivity (Too many technologies to explore, not much time available)

Additional FAQs

6. Windows - Sysadmin, reboot etc.

12

Abbildung 23: sed mit Muster- und Zeilenadressen. Quelle: Kommandos grep sed(1).pdf, Folie 12.

8.4 Zeilen löschen und alternative Trennzeichen

```
sed -E '1d' datei # erste Zeile  
sed -E '7,9d' datei # Zeilen 7 bis 9  
sed -E '/^$/d' datei # vollständig leere Zeilen  
sed -E '/[[:space:]]an[[:space:]]/d' zuege  
sed -E 's#Linz/Donau Hbf#Linz Hbf#g' zuege
```

sed – Zeilen löschen: Beispiele

Beispiel 6: Entferne alle Leerzeilen aus einem Dokument

```
sed -r '/^$/d' mydocument
```

Beispiel 7: Entferne die erste Zeile eines Dokuments bzw. lösche den Zeilenbereich 7 bis 9.

```
sed -r '1d' mydocument
```

```
sed -r '7,9d' filename.txt
```

16

Abbildung 24: Zeilen mit sed löschen. Quelle: Kommandos grep sed(1).pdf, Folie 16.

Das Trennzeichen nach s muss nicht / sein. Bei Pfaden oder Datumsangaben sind #, | oder ; oft lesbarer und sparen viele Backslashes.

8.5 Gierige Muster entschärfen

.^{*} ist gierig: Es nimmt so viele Zeichen wie möglich. In einer einzelnen Zeile reicht <.^{*}> daher von der ersten passenden < bis zur letzten passenden >. <[[^]>]^{*}> begrenzt den Treffer auf die nächste schließende Klammer. Das eignet sich nur für einfache, einzeilige Beispiele; allgemeines HTML sollte mit einem HTML-Parser verarbeitet werden.

Gierige und genügsame „regexs“

- Abhilfe: Um diese gierige regex genügsamer zu machen, gibt's folgende Möglichkeit.

```
sed -r 's/<[^>]*>//g' myhtml
```

Erklärung:

Der Ausdruck `<.*>` wird entschärft:

Die schließende spitze Klammer wird beim Matchen auf eine beliebige Zeichenfolge ausgeschlossen und kann daher nicht gefunden werden.

Das erste Auftreten des Zeichens „>“ beendet die Suche!

15

Abbildung 25: Entschärfung eines gierigen Musters. Quelle: Kommandos grep sed(1).pdf, Folie 15.

```
sed -E 's/<[^>]*>//g' myhtml | sed -E '/^$/d'
```

8.6 Variablen mit sed bearbeiten

```
alt='Löschfahrzeug'  
neu=$(printf '%s\n' "$alt" | sed -E 's/ö/oe/g')  
echo "$neu"
```

Originalaufgabe 8.1: Fortsetzung Zugverkehr mit sed

Quelle: Regexp_grep_sed_Übung.pdf, Seite 2. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Verwenden Sie den File zuege aus Originalaufgabe 7.1.

f) Löschen Sie die erste Zeile des Files.

g) Ändern Sie die Datumsangabe von xx.05.2015 auf xx.Mai 2016.

h) Löschen Sie alle Zeilen, die Ankunftszeiten enthalten.

i) Ersetzen Sie Linz/Donau Hbf durch Linz Hbf.

Originalaufgabe 8.2: Praktische Übungen mit sed

Quelle: ÜBUNGSAUFGABEN.pdf, Seite 2. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Verwenden Sie für die Übungen den File besucher.

1) Löschen Sie die erste Zeile und alle Leerzeilen.

2) Ersetzen Sie die Zeichenfolge ' - - ' durch ein einzelnes Leerzeichen.

3) Löschen Sie die Zahlen am Ende jeder Zeile.

4) Ersetzen Sie in jeder Zeile, die mit ics2f beginnt, die Zahl 06 durch 16.

5) Ersetzen Sie in den Zeilen 1 bis 5 die Zahl 06 durch 16.

6) Löschen Sie alle Zeilen, die mit modem05.pbnet.com.br beginnen.

- 7) Löschen Sie alle Zeilen zwischen dem ersten Auftreten von modem05.pbnet.com.br und Zeile 10.
- 8) Ersetzen Sie alle Datumsangaben 06/Nov/1997 durch 6.11.1997.

[Zurück zum Inhaltsverzeichnis](#)

9 Werkzeugkasten Textverarbeitung

9.1 cut und tr

cut wählt Felder oder Zeichenbereiche aus. Bei -d wird ein einzelnes Trennzeichen angegeben; -f wählt Felder. tr ersetzt oder löscht einzelne Zeichen. Für komplizierte Muster ist sed besser, für einfache Zeichenumsetzungen ist tr kürzer.

```
cut -d';' -f1,3 daten.csv
cut -f6 zuege.txt # Standard-Trenner: Tab
tr ',' ';' < eingabe > ausgabe
tr -d '\015' < windows.txt > linux.txt # löscht jedes CR-Zeichen
```

9.2 sort und uniq

```
sort namen.txt
sort -n zahlen.txt
sort -k2,2n daten.txt
sort namen.txt | uniq
sort namen.txt | uniq -c | sort -nr
```

uniq zählt nur benachbarte gleiche Zeilen. Deshalb steht davor fast immer sort. Mit sort -u kann Sortieren und Duplikatentfernung kombiniert werden.

9.3 head, tail und wc

Befehl	Zweck	Beispiel
head -n N	erste N Zeilen	head -n 5 datei
tail -n N	letzte N Zeilen	tail -n 1 datei
wc -l	Zeilenumbrüche zählen; bei korrekt abgeschlossenen Textzeilen die Zeilenzahl	wc -l < datei
wc -w	Wörter zählen	wc -w datei
wc -c	Bytes zählen	wc -c < datei

9.4 basename, dirname und Parametererweiterung

```
pfad='/home/user/report.txt'
basis=$(basename "$pfad") # report.txt
verz=$(dirname "$pfad") # /home/user
name=${basis%.txt} # report
endung=${basis##*.} # txt
```

9.5 Windows- und Linux-Zeilenenden

Linux verwendet LF, Windows traditionell CRLF. Das zusätzliche CR kann Muster am Zeilenende stören und als ^M sichtbar werden. In Bash prüft \$'\r\$' gezielt auf ein CR unmittelbar vor dem Zeilenende. Die Rückgabewerte von grep und dos2unix müssen getrennt geprüft werden.

```
grep -q $'\r$' "$datei"
status=$?
if [ "$status" -eq 0 ]; then
command -v dos2unix >/dev/null 2>&1 || { echo "dos2unix fehlt" >&2; exit 1; }
dos2unix "$datei" >/dev/null 2>&1 || { echo "Konvertierung fehlgeschlagen" >&2; exit 1; }
elif [ "$status" -eq 1 ]; then
echo "Die Datei enthält keine CRLF-Zeilenenden."
else
echo "Fehler beim Lesen der Datei" >&2; exit 1
fi
```

9.6 Beispiel einer vollständigen Pipeline

Aufgabe: Aus einer durch Doppelpunkt getrennten Datei sollen die unterschiedlichen Nahrungsmittelnamen ohne Mengenwort alphabetisch sortiert und gezählt werden.

```
cut -d':' -f1 essensplan.txt \  
| sed -E 's/^[^[:space:]]+[:space:]]+//' \  
| sort -u \  
| tee nmittel.txt \  
| wc -l
```

sort -u sortiert und entfernt Duplikate. tee schreibt die eindeutigen Namen in eine Datei und reicht sie gleichzeitig an wc -l weiter. Da diese Pipeline ihre Ausgabezeilen selbst mit Zeilenumbruch erzeugt, liefert wc -l hier zuverlässig die Anzahl.

Originalaufgabe 9.1: E-Mail-Adressen aus einer Studierendenliste

Quelle: ÜBUNGSAUFGABEN.pdf, Seite 3. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Der File studenten enthält Matrikelnummer, Nachname und einen oder mehrere Vornamen.

a) Erstellen Sie ein Shellskript email, das eine Datei stud_email erzeugt. Schema: 1410307047

S1410307047@students.fh-hagenberg.at Aberl David.

b) Erweitern Sie das Skript um eine zweite Adresse nach dem Schema Vorname.Name@students.fh-hagenberg.at. Beispiel: David.Aberl@students.fh-hagenberg.at.

c) Übergeben Sie den File studenten als Argument. Prüfen Sie, ob überhaupt ein Filename übergeben wurde und ob die Datei existiert. Beenden Sie das Skript andernfalls mit Fehlermeldung.

1410307047 Aberl David

1410307048 Aigner Martin Engelbert

1410307104 Amadi Christoph Chukwuka

1410307050 Baumgartner David

[Zurück zum Inhaltsverzeichnis](#)

10 Prüfungsmuster und vollständiges Beispiel

10.1 Eine Aufgabenstellung in kleine Schritte zerlegen

1. Aufruf und gewünschte Argumente markieren.
2. Alle Prüfungen am Anfang sammeln: Anzahl, Datei/Verzeichnis, Leserechte, Format.
3. Eingabeformat und Trennzeichen notieren.
4. Pro Ausgabe festlegen: neu schreiben oder anhängen?
5. Entscheiden, ob eine Pipeline oder eine while-read-Schleife klarer ist.
6. Zähler und Summen vor der Schleife initialisieren.
7. Am Ende sortieren, zählen und Ausgabetext exakt wie gefordert erzeugen.
8. Skript mit Leerzeichen in Dateinamen, leerer Datei und fehlenden Argumenten gedanklich testen.

Originaler Kurztest: tagesbedarf.sh

Quelle: Alter Kurztest(1).pdf, Seite 1. Aus der Originalaufgabe übernommen; für das Lernskript teilweise gekürzt und typografisch bereinigt.

Im File essensplan.txt stehen Mengenangabe, Nahrungsmittel und Kalorienzahl. Mengenangabe und Nahrungsmittel sind durch ein Leerzeichen getrennt; vor der Kalorienzahl steht ein Doppelpunkt.

Aufruf: ./tagesbedarf.sh essensplan.txt gesamtTagesbedarf.txt nmittel.txt

Prüfen Sie die Anzahl der Argumente und ob essensplan.txt existiert. Prüfen Sie außerdem, ob die Datei CRLF enthält; in diesem Fall ist sie mit dos2unix zu überschreiben.

In gesamtTagesbedarf.txt soll zuerst das Dateiformat und anschließend die Summe aller Kalorien ausgegeben werden.

Erstellen Sie zusätzlich eine alphabetisch sortierte Liste der Nahrungsmittel und geben Sie deren Gesamtanzahl aus.

Scheibe Vollkornbrot:180

Stück Butter:50

Stück Lachs:170

Stück Kartoffel:300

Portion Aufstriche:270

Stück Semmel:80

10.2 Robuste und geprüfte Musterlösung

```
#!/bin/bash
```

```
die() { printf 'Fehler: %s\n' "$*" >&2; exit 1; }
```

```
[ "$#" -eq 3 ] || die "Aufruf: $0 essensplan.txt gesamt.txt nmittel.txt"
```

```
eingabe=$1
```

```
gesamt=$2
```

```
nmittel=$3
```

```
if [ ! -f "$eingabe" ] || [ ! -r "$eingabe" ]; then
```

```
die "$eingabe ist keine lesbare reguläre Datei."
```

```
fi
```

```
if [ "$eingabe" = "$gesamt" ] || [ "$eingabe" = "$nmittel" ] || [ "$gesamt" = "$nmittel" ]; then
```

```
die "Ein- und Ausgabedateien müssen verschieden sein."
```

```
fi
```

```
grep -q $'\r$' "$eingabe"
```

```
case $? in
```

```

0) command -v dos2unix >/dev/null 2>&1 || die "dos2unix ist nicht installiert."
dos2unix "$eingabe" >/dev/null 2>&1 || die "CRLF-Konvertierung fehlgeschlagen." ;;
1) ;;
*) die "Das Dateiformat von $eingabe konnte nicht geprüft werden." ;;
esac
grep -q $'\r' "$eingabe"
case $? in
0) die "Die Datei enthält weiterhin CR-Zeichen." ;;
1) format="Die Datei beinhaltet keine CR!" ;;
*) die "Die Datei konnte nach der Konvertierung nicht geprüft werden." ;;
esac

tmp=$(mktemp) || die "Temporäre Datei konnte nicht erstellt werden."
trap 'rm -f "$tmp"' EXIT
muster='^(Stück|Scheibe|Portion) ([^[:space:]]*[^[:space:]]?)([0-9]+)$'
summe=0
while IFS= read -r zeile || [ -n "$zeile" ]; do
[[ $zeile =~ $muster ]] || die "Ungültige Eingabezeile: $zeile"
nahrungsmittel=${BASH_REMATCH[2]}
kcal=${BASH_REMATCH[4]}
summe=$((summe + 10#$kcal))
printf '%s\n' "$nahrungsmittel" >> "$tmp" || die "Temporäre Datei ist nicht beschreibbar."
done < "$eingabe"

sort -u "$tmp" > "$nmittel" || die "$nmittel kann nicht geschrieben werden."
anzahl=$(wc -l < "$nmittel") || die "$nmittel konnte nicht gezählt werden."
{
printf '%s\n' "$format"
printf 'Die Gesamtanzahl an Kalorien beträgt: %d kcal\n' "$summe"
printf 'Es befinden sich %d verschiedene Nahrungsmittel in der Datei %s.\n' "$anzahl" "$eingabe"
} > "$gesamt" || die "$gesamt kann nicht geschrieben werden."

```

10.3 Warum diese Lösung funktioniert

Block	Zweck
die()	gibt Fehlermeldungen einheitlich auf stderr aus und beendet das Skript mit Status 1.
Argument- und Dateiprüfung	verlangt genau drei Argumente, eine lesbare reguläre Eingabedatei und verschiedene Dateinamen.
grep + case	unterscheidet CRLF-Treffer (0), keinen Treffer (1) und Lesefehler (>1); eine zweite Prüfung stellt sicher, dass anschließend überhaupt kein CR-Zeichen mehr vorhanden ist.
mktemp + trap	verhindert eine teilweise Nahrungsmittelliste bei fehlerhaften Daten und löscht die temporäre Datei beim Beenden.
while read	liest zunächst die vollständige Zeile und verarbeitet auch eine letzte Zeile ohne abschließendes LF.
muster, [[... =~ ...]] und BASH_REMATCH	prüfen die gesamte Zeile: eines der drei Mengenwörter, genau ein Trennleerzeichen, genau einen Doppelpunkt, einen nicht leeren Namen ohne Rand-Whitespaces und eine Ziffernfolge; BASH_REMATCH übernimmt Name und Kalorienzahl.
10#\$kcal	erzwingt dezimale Interpretation, auch wenn eine Zahl mit 0 beginnt.
\$nahrungsmittel und sort -u	verwenden den aus der Regex extrahierten Namen, sortieren alphabetisch und zählen jedes Nahrungsmittel nur einmal.
wc -l und { ...; } >	zählen die erzeugten, newline-terminierten Listeneinträge und schreiben die Gesamtausgabe gemeinsam in die Zieldatei.

10.4 Mögliche separate grep- und sed-Teile

```

# grep: alle RJ- und WB-Zugnummern extrahieren und numerisch sortieren
grep -oE '(RJ|WB)[[:space:]]+[0-9]{3}' zuege.txt | sort -k2,2n

# sed: Linz/Donau Hbf ersetzen und Ankunftszeilen löschen
sed -E 's#Linz/Donau Hbf#Linz Hbf#g; /[[:space:]]an[[:space:]]/d' zuege.txt

```

10.5 Häufige Fehler in der Prüfung

- **Leerzeichen bei Tests vergessen:** ["\$#" -ne 3] ist richtig; ["\$#" -ne 3] ist falsch.
- **Variablen nicht gequotet:** immer "\$datei", besonders bei Dateinamen.

- **Ausgabedatei in jeder Schleifenrunde mit > überschrieben:** einmal vor der Schleife leeren, danach >>.
- **Regex nicht gequotet:** grep -E '...' verwenden.
- **BRE und ERE gemischt:** bei |, +, ?, { } und () am besten grep -E bzw. sed -E.
- Zeilen mit for word in \$(cat file) gelesen: für ganze Zeilen while IFS= read -r zeile || [-n "\$zeile"] verwenden.
- **uniq ohne sort:** uniq erkennt nur benachbarte Duplikate.
- **sed-Ausgabe nicht gespeichert:** sed ändert ohne -i nur stdout; bei Bedarf > neue_datei.
- **Statuscode zu spät geprüft:** \$? bezieht sich immer nur auf den unmittelbar vorherigen Befehl.

Letzter Prüfungsscheck: bash -n skript.sh prüft nur die Syntax. Danach das Skript mit LF- und CRLF-Datei, einer letzten Zeile ohne LF, doppelten Nahrungsmitteln, einer fehlerhaften letzten Trennzeichenzeile, fehlenden Argumenten und einer ungültigen Kalorienzahl ausführen; Ausgaben mit cat bzw. nl -ba kontrollieren.

[Zurück zum Inhaltsverzeichnis](#)

A Kompakte Befehlsübersicht

A.1 Shell und Tests

Aufgabe

Argumentzahl prüfen
Datei prüfen
Verzeichnis prüfen
String nicht leer
Zahl vergleichen
Fehlermeldung
Skript beenden
Integer rechnen
Kommandoausgabe speichern

Muster

```
[ "$#" -eq 2 ]  
[ -f "$1" ]  
[ -d "$1" ]  
[ -n "$x" ]  
[ "$a" -lt "$b" ]  
echo "Fehler" >&2  
exit 1  
summe=$((summe + wert))  
wert=$(kommando)
```

A.2 grep

Zweck

Trefferzeilen
Anzahl Trefferzeilen
Nur Matches
Nicht passende Zeilen
Nur Status
Fester Text

Muster

```
grep -E 'regex' datei  
grep -Ec 'regex' datei  
grep -oE 'regex' datei  
grep -Ev 'regex' datei  
grep -qE 'regex' datei  
grep -F 'text' datei
```

A.3 sed

Zweck

erstes Vorkommen ersetzen
alle Vorkommen ersetzen
nur passende Zeilen
Zeilenbereich
Zeile löschen
Musterzeilen löschen
anderer Trenner

Muster

```
sed -E 's/alt/neu/' datei  
sed -E 's/alt/neu/g' datei  
sed -E '/muster/s/alt/neu/g' datei  
sed -E '1,5s/alt/neu/g' datei  
sed -E '1d' datei  
sed -E '/muster/d' datei  
sed -E 's/#alt#neu#g' datei
```

A.4 Regex-Spickzettel (ERE)

Muster

```
^ / $  
.  
[abc] / [^abc]  
[0-9] / [[:digit:]]  
[[:space:]]  
* / + / ?  
{n} / {n,m}  
(...)  
A|B  
\\.
```

Bedeutung

Anfang / Ende der Zeile
beliebiges einzelnes Zeichen
ein Zeichen aus / nicht aus der Menge
Ziffer
Whitespace
0..n / 1..n / 0..1
genau n / zwischen n und m
Gruppe
Alternative
echter Punkt

[Zurück zum Inhaltsverzeichnis](#)

B Quellenübersicht

Das Lernskript fasst die folgenden bereitgestellten PDFs zusammen. Inhaltlich doppelte Folienversionen wurden zusammengeführt; Abbildungen stammen aus den jeweils genannten Folienseiten.

- Alter Kurztest(1).pdf
- Angabe bplist - Beispiel 3(1).pdf
- Kommandos grep sed(1).pdf
- Rechnen in der Shell.pdf
- Regexp_grep_sed_Übung.pdf
- Reguläre Ausdrücke.pdf
- Shellprogrammierbeispiele.pdf
- Shell_Programmierung_Verwendung_Schleifen.pdf
- Shellprogrammierung_wetter.pdf
- Slides - Basics (WH).pdf
- Slides - Intro.pdf
- T2_RegEx_Übung.pdf
- T4_Regular_Expressions.pdf
- T4_shell_basics.pdf
- T5_shell_programming.pdf
- ÜBUNGSAUFGABEN.pdf
- Vordefinierte Character-Klassen für grep sed.pdf

Die Aufgabenblöcke stammen aus den Originalunterlagen, wurden für das Lernskript jedoch teilweise gekürzt und typografisch vereinheitlicht. Die Erklärungen, Korrekturhinweise, robusten Varianten und die geprüfte Musterlösung wurden für dieses Lernskript ergänzt.

[Zurück zum Inhaltsverzeichnis](#)